Amrita School of Computing 22AIE457

Full stack Development

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

(Using MERN Stack)

(Subject to necessary edits in given format based on project topic/requirement)

For: Project Name Version: 1.0 Date: 05/03/2026

Objective:

To clearly document all functional and non-functional requirements of the system, ensuring a shared understanding among stakeholders and serving as a reference for design, development, testing, and maintenance.

Team Information:

Team Member Name	Student Roll No	Project Title
Adithya Varma Kusampudi	AM.SC.U4AIE23008	FoundersForge – LLM powered Collaboration and Investment Platform
Velamala Pavan Krishna	AM.SC.U4AIE23063	

1. Introduction

1.1 Purpose

The purpose of this Software Requirements Specification (SRS) is to define the requirements for FoundersForge, an LLM powered collaboration and investment platform developed using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system aims to connect founders, contributors, and investors within a unified platform where founders can present project ideas, contributors can showcase their skills through parsed resumes, and investors can discover and support promising projects. The platform integrates AI assisted project post structuring, semantic search using embeddings and Retrieval Augmented Generation and project specific communication tools to improve collaboration and project discovery. Additionally, it provides investment tracking dashboards to help investors monitor funded projects and founders track financial involvement. This document serves as a reference for developers, testers, and stakeholders to guide the design, development, and evaluation of the system.

1.2 Document Conventions

The following conventions are used in this document:
FR – Functional Requirement
NFR – Non-Functional Requirement
API – Application Programming Interface
UI – User Interface
DB – Database
LLM – Large Language Models
RAG – Retrieval Augmented Generation
JWT – JSON Web Token

All functional requirements are labeled using FR identifiers, while non-functional requirements are labeled using NFR identifiers

1.3 Intended Audience

- **Founders:** to understand how they can submit project ideas, form teams, collaborate with contributors, and track investments for their projects.

- **Contributors:** to understand how they can upload resumes, showcase their skills, discover projects, and collaborate with founders.

- **Investors:** to understand how they can explore projects, evaluate opportunities, and track their investments within the platform.

1.4 Product Scope

FoundersForge is an LLM-powered collaboration and investment platform that connects founders, contributors, and investors within a single ecosystem. The platform addresses the challenge that founders often have innovative ideas but lack the right team members and funding, while contributors and investors struggle to find relevant and meaningful opportunities. Unlike traditional platforms that focus only on freelancing, networking, or funding, FoundersForge integrates idea sharing, team formation, collaboration, and investment tracking into one unified system.

The platform allows founders to submit raw project ideas, which are automatically structured into professional project descriptions using LLM technology. Contributors can upload resumes that are parsed into structured skill profiles to enable intelligent matching with suitable projects. Investors and contributors can discover projects through a semantic search system powered by vector embeddings and Retrieval-Augmented Generation (RAG), ensuring more relevant discovery.

FoundersForge also provides project-specific communication through chat rooms that allow founders, contributors, and investors to collaborate and discuss ideas in real time. In addition, an investment tracking dashboard enables investors to monitor funded projects while founders can track financial contributions and progress.

The system follows a **client–server architecture**, where the frontend web application enables users to create profiles, discover projects, and collaborate. The backend server manages user accounts, project data, communication services, and AI processing. An AI layer handles project formatting, resume parsing, and semantic search using LLMs and embedding models, supported by a database and vector store for structured data and semantic retrieval.

By integrating AI-powered structuring, intelligent search, collaboration tools, and investment tracking, FoundersForge aims to simplify project discovery, improve team formation, and create a more efficient environment for innovation and funding.

1.5 References

IEEE 830 SRS Standard
MERN Stack Documentation
React Documentation
Node.js Documentation
MongoDB Documentation

2. Overall Description

2.1 Product Perspective

FoundersForge is a web-based collaboration and investment platform built using the MERN stack (MongoDB, Express.js, React.js, Node.js) and enhanced with AI capabilities. The system acts as a centralized ecosystem that connects founders, contributors, and investors, enabling idea sharing, team formation, project discovery, and investment tracking within a single platform.

The system follows a multi-layer client–server architecture consisting of the following components:

Presentation Layer

Implemented using React.js, this layer provides the user interface through which founders, contributors, and investors interact with the platform. It allows users to create accounts, submit project ideas, upload resumes, search for projects, communicate through chat rooms, and view dashboards for collaboration and investment tracking.

Business Logic Layer

Implemented using Node.js and Express.js, this layer handles application logic, user authentication, project management, resume processing, communication services, and investment tracking. It also manages API requests from the frontend and coordinates interactions between the AI services and the database.

AI Processing Layer

This layer integrates Large Language Models (LLMs) and embedding models to provide intelligent system capabilities. It is responsible for formatting raw project ideas into structured posts, parsing contributor resumes into skill profiles, and enabling semantic project discovery using Retrieval-Augmented Generation (RAG) and vector embeddings.

Data Layer

Implemented using MongoDB and a vector database, this layer stores structured system data such as user profiles, project information, resumes, messages, and investment details. The vector store maintains semantic embeddings that support intelligent search and recommendation functionality.

The system communicates internally through RESTful APIs over HTTP/HTTPS, allowing efficient interaction between the frontend, backend services, and AI components. External AI services or model APIs may also be integrated to support LLM-based processing and semantic search capabilities through secure backend API calls.

2.2 Product Functions

The FoundersForge system provides the following core functions:

User Management
• User registration and login
• Secure authentication and account management
• Profile creation for founders, contributors, and investors
• Profile updates including skills, interests, and project preferences

Founder Services
• Submit raw project ideas to the platform
• Automatically structure project descriptions using LLM assistance
• Publish and manage project posts
• Search for contributors and potential collaborators
• Communicate with contributors and investors through project chat rooms
• Track funding and financial participation through investment dashboards

Contributor Services
• Upload resumes to the platform
• Automatic resume parsing into structured skill profiles using AI
• Discover projects using intelligent semantic search
• Apply or express interest in projects
• Collaborate with founders and team members through project chat rooms

Investor Services
• Discover promising projects through semantic search and recommendations
• View structured project descriptions and project progress
• Invest in projects and support founders
• Monitor funded projects through investment tracking dashboards

AI Powered Platform Features
• LLM based structuring of raw project ideas into professional project posts
• Resume parsing and extraction of contributor skills and experience
• Intelligent semantic search using vector embeddings
• RAG to improve project discovery and recommendations

Collaboration and Communication
• Project specific chat rooms for founders, contributors, and investors
• Real time communication and discussion about project progress
• Shared interaction space for team coordination

System and Data Management
• Storage and management of user profiles, projects, and investment data
• Semantic embedding storage for intelligent search
• Secure handling of platform data and communication between system components

2.3 User Classes and Characteristics

Founder
• Registers and creates a profile on the platform
• Submits project ideas that are structured using AI
• Manages project posts and collaborates with contributors
• Tracks project funding and investor involvement

Contributor
• Creates a profile and uploads a resume
• Resume is parsed into a structured skill profile using AI
• Searches and applies for projects
• Collaborates with founders through project communication tools

Investor
• Creates an investor profile on the platform
• Searches and evaluates available projects
• Invests in promising projects
• Tracks investments through project dashboards

Admin
• Manages users and platform activities
• Monitors projects and system usage
• Maintains proper functioning of the platform

Guest User
• Can browse public project listings
• Cannot create projects, apply to projects, or invest without registering

2.4 Operating Environment

Client Side - Web browser (Chrome, Firefox, Safari, Edge)

Server Side - Node.js runtime environment

Database - MongoDB (local/cloud)

2.5 Design and Implementation Constraints

• The system will be built using the MERN.
• User authentication will be implemented using JWT.
• The backend will be designed following a RESTful API architecture.
• The user interface should be responsive and accessible on both mobile and desktop devices.
• The platform must ensure secure storage and protection of user data.

2.6 Assumptions and Dependencies

• Users are expected to have a stable internet connection to access the platform.
• The platform will be accessed through modern web browsers.
• MongoDB and external AI services/APIs will be available to support system functionality.

3. System Features (Functional Requirements)

3.1 User Registration (FR1)

The system shall allow new users to create an account on the platform.

Users shall provide the following information during registration:

• Full name
• Email address
• Password
• User role (Founder, Contributor, or Investor)

The system shall validate that the email address is unique before creating the account.

The system shall securely store passwords using encryption or hashing techniques.

After successful registration, the user shall be able to log in and access the platform according to their selected role.

3.2 User Login (FR2)

The system shall allow registered users to log in using their email address and password.

Upon successful authentication:

• The system shall generate a JWT authentication token.
• The user shall be redirected to their respective dashboard.

If authentication fails, the system shall display an appropriate error message indicating invalid credentials.

The system shall maintain secure session management to protect user accounts.

3.3 User Profile Management (FR3)

The system shall allow registered users to create and manage their personal profiles.

Users shall be able to:

• Update personal details
• Add profile descriptions
• Upload profile pictures
• Manage role-related information

Contributor profiles shall contain skill information extracted from resumes.

Founder profiles shall contain project information and activity history.

Investor profiles shall contain funding activity and supported projects.

3.4 Founder Project Submission and Structuring (FR4)

The system shall allow founders to submit raw project ideas through the platform.

Founders shall be able to:

• Enter project title or idea description
• Provide optional additional information such as project goals or requirements

The system shall send the submitted project description to an LLM processing module.

The LLM shall convert the raw input into a structured project post containing:

• Project title
• Problem description
• Proposed solution
• Required skills or roles
• Project objectives

After processing, the structured project information shall be stored in the database.

The system shall generate vector embeddings for the project data and store them in the vector database to support semantic search.

3.5 Contributor Resume Upload and Parsing (FR5)

The system shall allow contributors to upload resumes in supported formats such as PDF or DOCX.

Once a resume is uploaded:

• The system shall send the document to the AI processing layer.
• The LLM shall extract structured information including:
• Skills
• Work experience
• Education
• Technologies used

The extracted data shall be stored in the system database as a structured contributor profile.

The system shall generate vector embeddings from the extracted information to support semantic matching with projects.

Contributors shall be able to review and update their extracted profile information if needed.

3.6 Semantic Project Discovery (RAG-Based Search) (FR6)

The system shall allow contributors and investors to discover projects using natural language search queries.

Users shall be able to search using descriptions such as:

• “AI project needing frontend developers”
• “Blockchain startup looking for investors”

The system shall process the query using embedding models and convert the query into a vector representation.

The system shall retrieve relevant projects from the vector database using semantic similarity matching.

The retrieved projects shall be ranked according to similarity scores.

The system shall present the results in a structured format showing:

• Project title
• Description summary
• Required skills
• Founder information

This feature shall improve project discovery by providing intelligent matching rather than simple keyword search.

3.7 Project Collaboration and Chat System (FR7)

The system shall create a **dedicated chat room for each project**.

The chat room shall allow founders, contributors, and investors to communicate and collaborate.

Users participating in a project shall be able to:

• Send text messages
• Discuss project details
• Ask questions related to the project

Messages shall be stored in the system database to maintain communication history.

The chat system shall support real-time communication using server-side communication services.

3.8 Project Application and Team Formation (FR8)

The system shall allow contributors to express interest or apply to projects.

Contributors shall be able to:

• View project requirements
• Submit applications to join projects

Founders shall be able to:

• Review contributor profiles
• Evaluate contributor skills and experience
• Accept or reject applications

Accepted contributors shall become project collaborators and gain access to project communication features.

3.9 Investment and Funding Management (FR9)

The system shall allow investors to support projects by providing funding.
Investors shall be able to:

• View project descriptions and progress details
• Review founder and team information
• Select projects they wish to support

The system shall record the following information for each investment:

• Investor identity
• Project funded
• Investment amount
• Date of transaction

The system shall store this information securely in the database.

3.10 Investment Tracking Dashboard (FR10)

The system shall provide dashboards for both founders and investors.
Investor dashboards shall display:

• List of funded projects
• Total investments made
• Investment history

Founder dashboards shall display:

• Total funds received
• Investor details
• Funding progress of their projects

This feature allows both parties to monitor financial activity transparently.

3.11 Authorization and Role-Based Access Control (FR11)

The system shall implement role-based access control to restrict platform functionality.

The system shall enforce the following permissions:

• **Founders** can create and manage their own projects.
• **Contributors** can upload resumes and apply to projects.

• **Investors** can browse and fund projects.
• **Admins** have full access to manage the platform.

Unauthorized users shall not be able to access restricted system features.

The system shall verify permissions before executing any protected operation.

4. External Interface Requirements

4.1 User Interface

The system shall provide a responsive web-based interface that allows founders, contributors, and investors to interact with the platform from both desktop and mobile devices.

The interface shall include the following main pages:

Landing Page
• Displays an overview of the CollabBridge platform
• Provides information about collaboration and investment features
• Includes login and registration options

Authentication Pages
• Registration form for new users
• Login form with input validation
• Secure access to user dashboards after authentication

User Dashboards
• Founder dashboard to submit and manage project ideas and track funding
• Contributor dashboard to upload resumes, view extracted skills, and discover projects
• Investor dashboard to explore projects and monitor investments

Project Collaboration Page
• Displays structured project details
• Provides project-specific chat rooms for communication between founders, contributors, and investors

4.2 Hardware Interface

The system does not require specialized hardware.

The platform can run on devices such as:

• Desktop computers
• Laptop computers
• Tablets and smartphones

Users only need a device capable of running a modern web browser with an internet connection.

4.3 Software Interface

The platform integrates multiple software components to support system functionality.

MongoDB Database
• Stores user profiles, project posts, resumes, messages, and investment records

Node.js and Express.js Backend
• Handles API requests and business logic
• Manages authentication, project management, and communication services

React.js Frontend
• Provides the user interface and dashboard functionality
• Communicates with backend services using REST APIs

AI and Vector Services
• LLMs used for structuring project ideas and parsing resumes
• Vector database used to store embeddings for semantic search and RAG-based discovery

4.4 Communication Interface

Communication between system components shall occur using standard web protocols.

• Frontend and backend communicate through HTTP/HTTPS using RESTful APIs
• Data exchanged between components is formatted using JSON
• Authentication is handled using JWT-based tokens
• Real-time project communication may use WebSocket connections for chat functionality

5. Non-Functional Requirements

5.1 Performance Requirements (NFR1)

The system shall provide efficient performance for all platform users.

• Common actions such as login, project browsing, and profile viewing shall respond within 3 seconds under normal conditions.
• The system shall support multiple users accessing the platform simultaneously without major performance issues.
• Database queries and semantic search operations shall be optimized for faster results.

5.2 Security Requirements (NFR2)

The system shall ensure secure access and protection of user data.

• User passwords shall be securely stored using bcrypt hashing.
• Authentication shall be implemented using JWT tokens.
• Only authorized users shall be allowed to access protected resources.
• All communication between client and server shall use HTTPS.

5.3 Reliability Requirements (NFR3)

The system shall operate reliably and maintain data integrity.

• The system shall handle invalid inputs and display appropriate error messages.
• Unauthorized access attempts shall be prevented.
• The system shall ensure that stored data is not lost during failures.

5.4 Usability Requirements (NFR4)

The system shall provide a simple and user-friendly interface.

• The interface shall be easy to navigate for founders, contributors, and investors.
• Users shall be able to perform tasks such as submitting projects, uploading resumes, and discovering projects easily.
• The interface shall be responsive on both desktop and mobile devices.

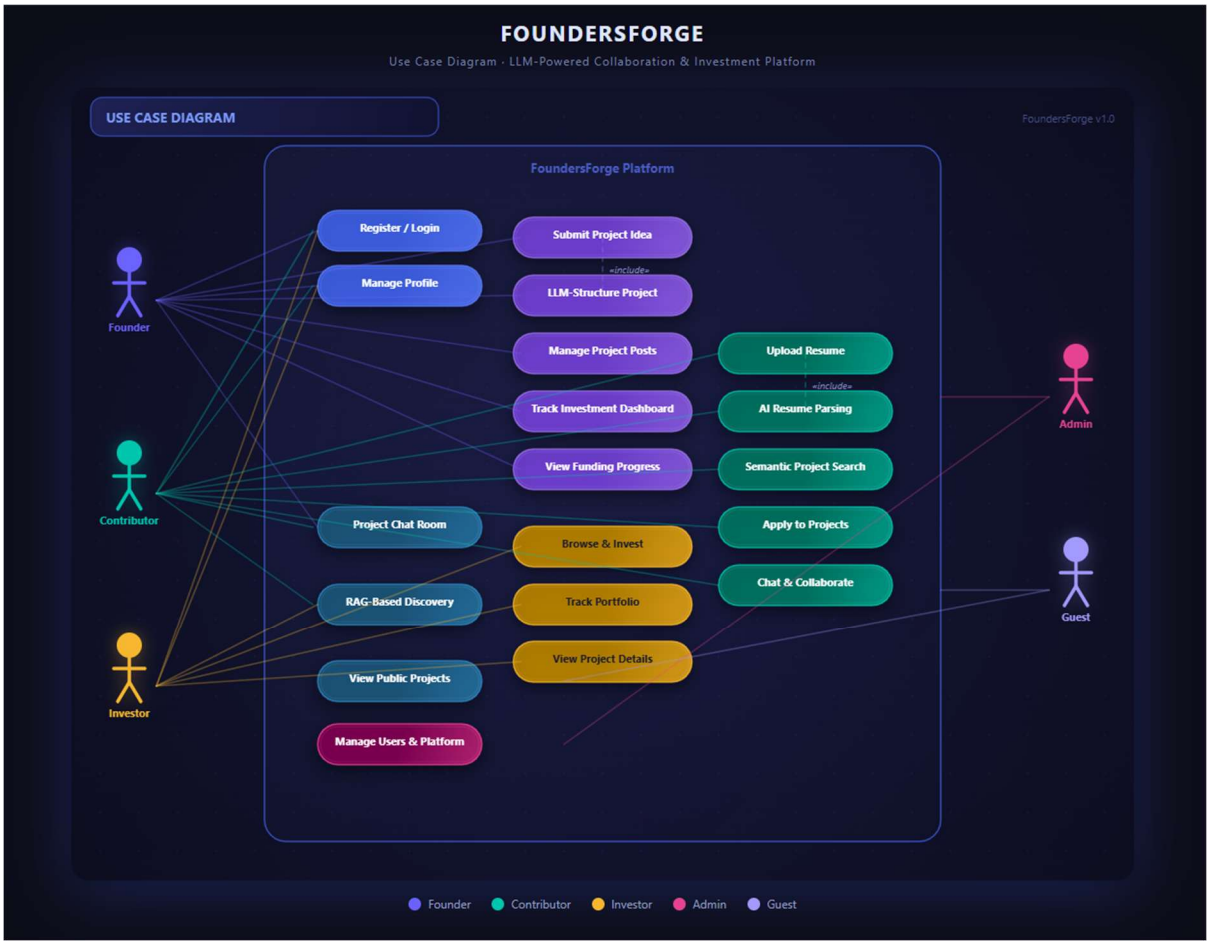
5.5 Maintainability Requirements (NFR5)

The system shall be designed for easy maintenance and future updates.

• The codebase shall follow a modular structure.
• The backend shall follow a clear API-based architecture.
• The system shall allow future enhancements such as additional AI features or integrations.

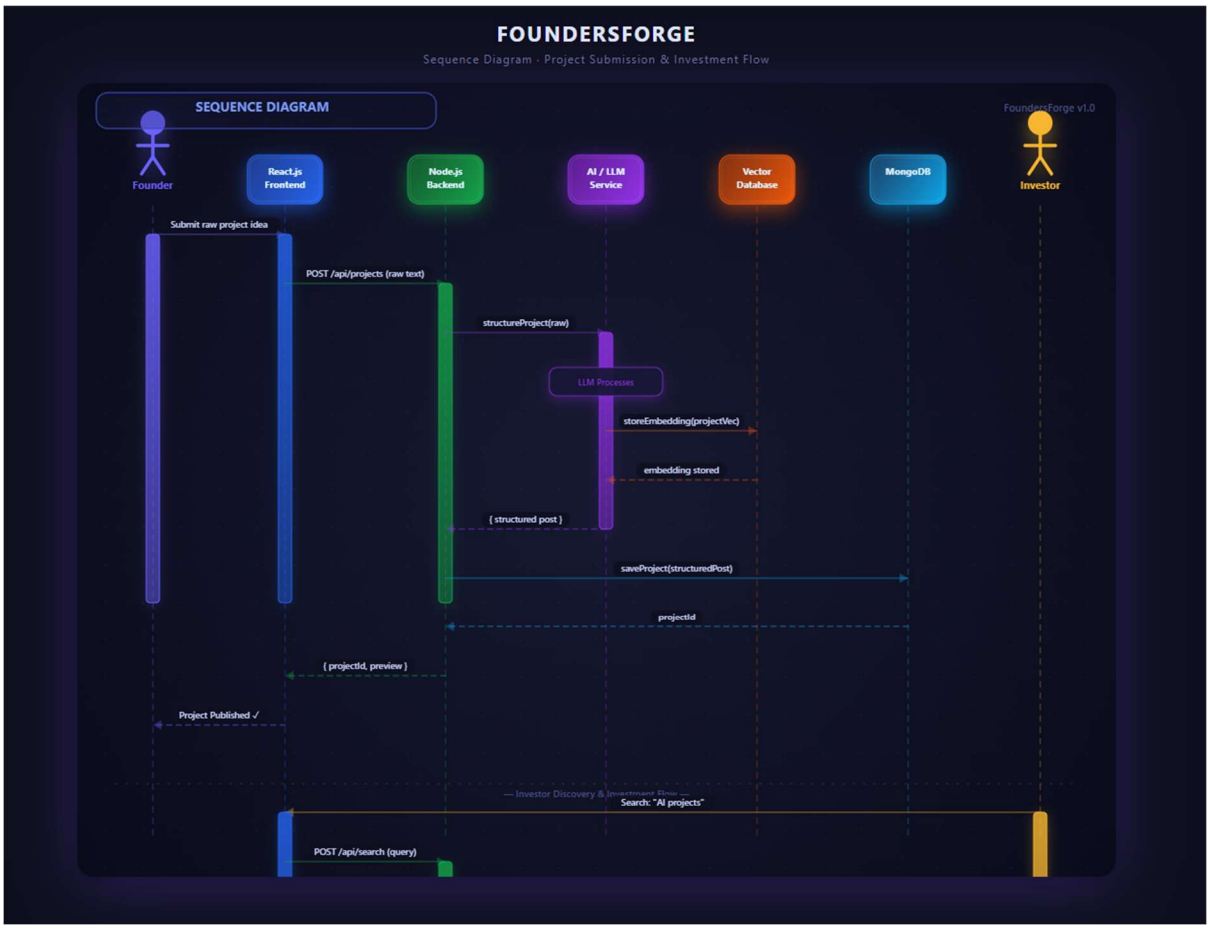
6. System Models

Use Case Diagram

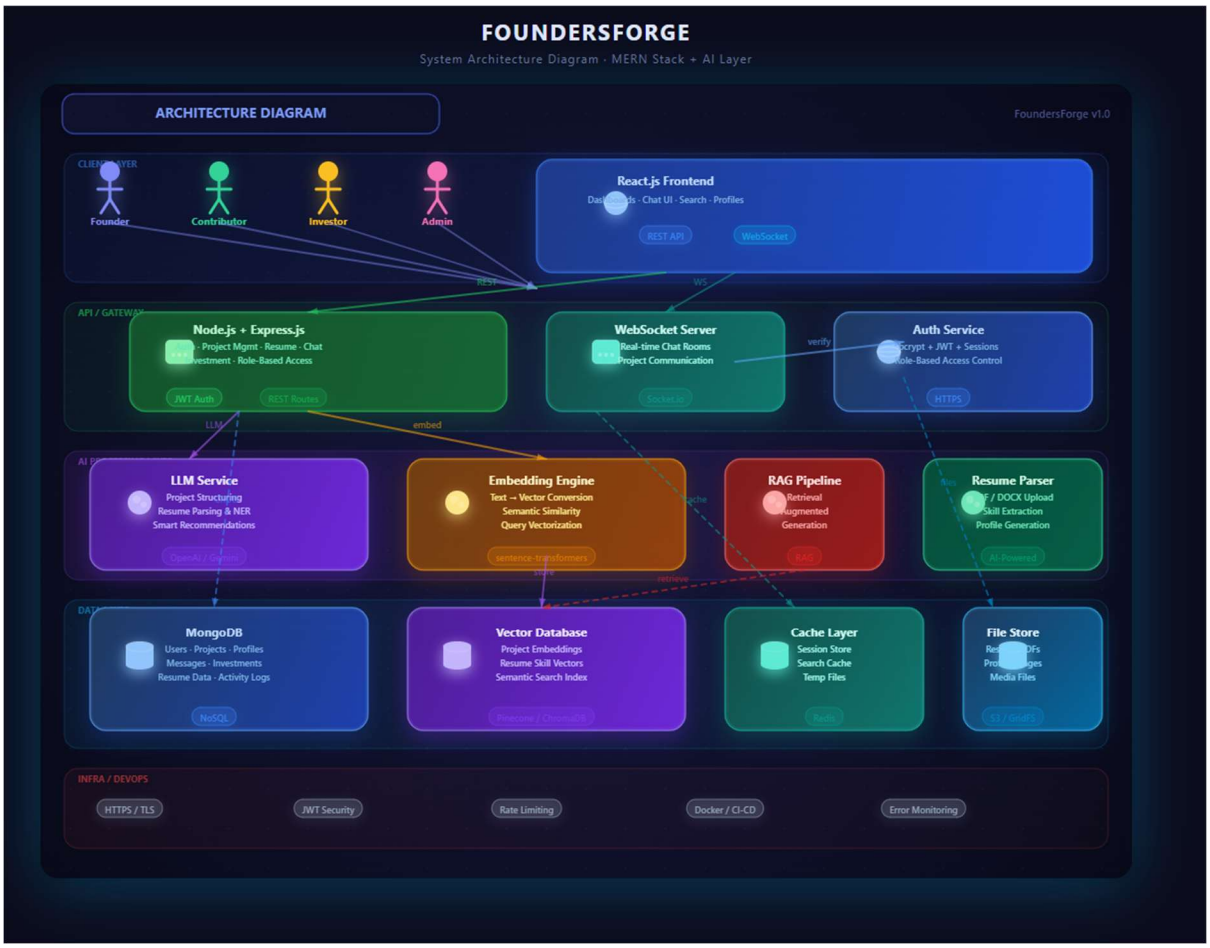


ER Diagram

Sequence Diagram



Architecture Diagram


FOUNDERSFORGE
System Architecture Diagram · MERN Stack + AI Layer
ARCHITECTURE DIAGRAM
FoundersForge v1.0
Founder
Contributor
Investor
Admin
React.js Frontend
REST API
WebSocket
REST
WS
Node.js + Express.js
JWT Auth
REST Routes
WebSocket Server
Real-time Chat Rooms
Project Communication
verify
Auth Service
HTTPS
LLM
embed
LLM Service
Project Structuring
Resume Parsing & NER
Smart Recommendations
Embedding Engine
Text → Vector Conversion
Semantic Similarity
Query Vectorization
sentence-transformers
RAG Pipeline
Retrieval
Augmented
Generation
RAG
Resume Parser
Skill Extraction
Profile Generation
retrieve
MongoDB
Users · Projects · Profiles
Messages · Investments
Resume Data · Activity Logs
NoSQL
Vector Database
Project Embeddings
Resume Skill Vectors
Semantic Search Index
Cache Layer
Session Store
Search Cache
Temp Files
Redis
File Store
Media Files
INFRA / DEVOPS
HTTPS / TLS
JWT Security
Rate Limiting
Docker / CI-CD
Error Monitoring